

```

#Install the ollama library

!pip install ollama

#Import the ollama library

import ollama

from pinecone import Pinecone
from dotenv import load_dotenv
from langchain.text_splitter import RecursiveCharacterTextSplitter
from pypdf import PdfReader
from tqdm import tqdm
import os
load_dotenv()
pinecone_api_key = os.getenv("PINECONE_API_KEY")
pc=Pinecone(api_key=pinecone_api_key)

#this is where we will load over pdf file

def load_pdf(pdf_path: str, chunk_size:int=500, chunk_overlap=200):
    """
    Load Pdf file and split it into chunks
    """

    print("Loading pdf file")
    pdf=PdfReader(pdf_path)
    text=""

    for page in tqdm(pdf.pages,desc="Extracting text"):
        text += page.extract_text()

    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        separators=["\n\n", "\n", ".", "!", "?", ",", " ", ""]
    )
    chunks = splitter.split_text(text)
    print(f"Created {len(chunks)} chunks")
    return chunks

#function for creating embeddings

def create_embeddings(chunks,model_name="llama3.1:latest"):
    embeddings=[]
    print("Generating embeddings")

    for chunk in tqdm(chunks):
        embedding=ollama.embeddings(
            model=model_name,
            prompt=chunk

```

```

    )
    embeddings.append(embedding['embedding'])
    print(f"📄 Generated {len(embeddings)} embeddings")
    return embeddings

#function to upload embeddings to pinecone

def
upload_to_pinecone(chunks, embeddings, index_name="buildingrag", batch_size=50):
    index=pc.Index(index_name)
    vectors=[]

    for i, (chunk, emb) in enumerate(zip(chunks, embeddings)):
        vectors.append(
            {
                "id": f"chunk{i}",
                "values": emb,
                "metadata": {"text": chunk}
            })

    print(f"📄 Upserting {len(vectors)} vectors to Pinecone...")
    for i in tqdm(range(0, len(vectors), batch_size)):
        batch = vectors[i:i+batch_size]
        index.upsert(vectors=batch)

    print("📄 Upload complete")
    return index

def test_retrieval(question, index, model_name="llama3.1:latest",
top_k=3):
    """Test retrieval from Pinecone"""
    print(f"\n📄 Question: {question}")

    q_embedding = ollama.embeddings(
        model=model_name,
        prompt=question
    )

    results = index.query(
        vector=q_embedding['embedding'],
        top_k=top_k,
        include_metadata=True
    )

    context = "\n".join([m['metadata']['text'] for m in
results['matches']])
    prompt = f"Context: {context}\n\nQuestion: {question}\n\nAnswer:"

```

```

response = ollama.chat(
    model=model_name,
    messages=[{"role": "user", "content": prompt}]
)

answer = response['message']['content']
print(f"👉 Answer: {answer}")
return answer

pdf_path=r"yourpdfpath"
chunks = load_pdf(pdf_path)

#to see how chunks looks like
chunks

#creating embeddings
resulting_embedding=create_embeddings(chunks)

#viewing how the embeddings looks like

resulting_embedding

#upserting data to pinecone
resulting_datatopinecone=upload_to_pinecone(chunks,resulting_embedding
)

# Test retrieval
index = pc.Index("yourindexname")    # replace with your index name
answer = test_retrieval("your input query?", index)

```